A Holding-Constraint Roadmap for the Two-Arm Bar Carry

A technical report on the PRM pilot (`holding_prm`) and its first measurements

husky_assembly_tamp `motion_planner` — written for Yijiang Huang

30 September 2026

Abstract

The task-space BiRRT that plans the M1 two-arm bar carry (`plan_constrained_dual_arm`; see `docs/cdfm_task_space_birrt_report.tex`) holds the bar exactly at every waypoint and loses most of its extensions to the 10-degree joint-continuity gate. This report describes the opposite planner, proposed in the meeting of 29 September 2026 and built as a pilot in `motion_planner/holding_prm/`: a probabilistic roadmap whose nodes hold the bar exactly, whose edges are straight lines in joint space (so the joints never jump), and whose edge cost is how far the two tool flanges drift from the bar along the line. The query searches from a *set* of start configurations to a *set* of goal configurations, so the start pose and both branch choices are chosen by the search rather than fixed beforehand. The pseudocode is written from the code; lines that differ from the BiRRT are printed in red and tagged [P-n], collected in Section 8. Section 10 reports the first measurements on bar B18, goal 2 (the goal whose 807 starts the BiRRT was run on in full) and on three goals of bar B3.

Contents

1 What is being planned, and what changes **2**

2 Where the code lives **3**

3 Building blocks **4**
- 3.1 The holding violation . . . 4
- 3.2 Straight lines in joint space, and why continuity is free . . . 4
- 3.3 Turns . . . 4
- 3.4 Collision . . . 5

4 Building the roadmap **5**
- 4.1 Anchors . . . 5
- 4.2 Sampling: forward kinematics for one arm, IK for the other . . . 5
- 4.3 Edges . . . 6

5 Querying the roadmap **7**
- 5.1 Endpoints . . . 7
- 5.2 The gap of every node . . . 7
- 5.3 Lazy collision checking . . . 7
- 5.4 The questions asked . . . 8

6 Running it in parallel **9**

7 Storage and the dashboard **9**

8 Core differences from the task-space BiRRT **10**

9 Parameters and their defaults **11**

10 What the pilot measured **11**
10.1 What was run . . . 11
10.2 How fast a joint-space line leaves the holding surface . . . 11
10.3 The roadmaps and what they cost . . . 12
10.4 Starts 0 and 12, and all 807 starts, against the node budget . . . 13
10.5 Which starts are near the goal: classes, carry anchors, and the BiRRT . . . 14
10.6 The best path . . . 15
10.7 Seeds and two sampler choices . . . 16
10.8 Bar B3 . . . 17
10.9 What this says, and what it does not . . . 18

11 Reading guide **18**

1 What is being planned, and what changes

The movement is the same as in the BiRRT report: both UR5e arms of the Husky hold one bar rigidly and carry it from a carry (home) pose to the approach pose in front of the structure; the base does not move. The notation is the same too: $q = (q_L, q_R) \in \mathbb{R}^{12}$ are the twelve arm joints; $G_L, G_R \in \mathrm{SE}(3)$ are the grasps (the pose of each `tool0` flange in the bar frame); FK_L, FK_R are the flange forward kinematics. A configuration holds the bar when both flanges agree on one bar pose,

$$\mathrm{FK}_L(q_L)\, G_L^{-1} \;=\; \mathrm{FK}_R(q_R)\, G_R^{-1}, \tag{1}$$

and the set $\mathcal{M}$ of such configurations is a six-dimensional surface in the twelve-dimensional joint space.

The two ways to treat the two constraints. A path has to satisfy two things at once: every configuration holds the bar (Eq. 1), and consecutive configurations are close in joint space (no joint moves more than $\tau = 10^\circ$ between waypoints). The BiRRT makes the first one exact and tests the second; its night-1 benchmark measured that 75 % of its extensions stop on the second. The roadmap swaps the roles:

	task-space BiRRT	holding roadmap (this report)
holding the bar	exact at every waypoint (IK)	exact at nodes; *measured* along edges
joint continuity	tested per waypoint (the gate)	true by construction (straight joint lines)
what a failure means	the tree stops growing	the edge costs more
answer	a path or nothing	the smallest holding error any path needs

The last row is the point of the pilot. The BiRRT cannot tell "no path exists" from "the planner did not find it". The roadmap always returns a number: the smallest tolerance on the holding error at which some start connects to some goal. A small number means near-feasible; a large one means the start and goal branch sheets are probably not connected.

Only the goal pose is given. The export fixes the goal bar pose T_g and nothing else: neither the start pose nor either configuration. The BiRRT must pick one start configuration and one goal configuration before it searches (its difference D-9), and a chain of heuristics does that (goal branch pairing with the home anchors, backward tracking). The roadmap does not need to choose:

- the **goal set** is every collision-free analytical branch pair at T_g (up to 8×8 left-right combinations), each on the library's turn and on the home turn (Section 3.3);
- the **start set** is every collision-free branch pair at every candidate start pose; the pilot takes the benchmark library's surveyed carry poses (807 for B18 goal 2), which are the three carry anchors (horizontal, vertical, on the back) with their offsets and roll/yaw variants;

- one search runs from a virtual *source*, joined at zero cost to every start node, to a virtual *sink*, joined at zero cost to every goal node. The cheapest path therefore chooses the start pose, the start branch pair and the goal branch pair together, and the report says which it chose.

Because the grasps are fixed per bar, $\mathcal{M}$ is the same surface for every goal of a bar: one roadmap per bar and scene can serve all of its goals, and a query only adds its own start and goal nodes.

2 Where the code lives

Function	File	Role
`holding_violation`, `violation_from_tools`	`holding.py`	the violation measure (Alg. 1)
`tool0_world`, `build_holding_model`	`holding.py`	numpy forward kinematics on the cached arm bases
`interpolate_confs`, `rebranch12`	`holding.py`	joint-space lines; moving a configuration onto a 2π turn
`draw_driver`, `lift_draw`, `sample_chunk`	`sampling.py`	the node sampler (Algs. 2, 3)
`arm_branches`	`sampling.py`	one arm's analytical IK on the cached base
`expand_bar_pose`	`sampling.py`	every collision-free branch pair at a bar pose (Alg. 6)
`make_collision_fn`	`sampling.py`	the production full-cell collision check
`candidate_pairs`, `evaluate_edge`	`roadmap.py`	nearest neighbours; measuring one edge (Alg. 4)
`edge_cost`, `Roadmap`	`roadmap.py`	the penalised weight; the node and edge tables
`minimax_to_goals`, `route_from`	`search.py`	every node's gap to the goal set (Alg. 7)
`build_graph`, `shortest_between_sets`	`search.py`	source/sink search (Dijkstra)
`lazy_search`, `path_report`	`search.py`	lazy collision checking (Alg. 8); path profiles
`WorkerPool`, `init_worker`, `task_*`	`parallel.py`	worker processes (Section 6)
`build_roadmap`, `query_roadmap`	`pilot.py`	the two drivers (Algs. 5, 9)
`save_roadmap`, `copy_dashboard`	`store.py`	files under `<root>/prm/`; the dashboard page
`main`	`scripts/prm_pilot.py`	command line: `build`, `query`, `export`, `info`
page	`benchmark/dashboard/prm.html`, `prm-viewer.js`	the roadmap viewer (Section 7)
tests	`benchmark/tests/test_holding_prm.py`	15 checks, including one that the package never imports the planner core

Paths without a folder are in `husky_assembly_tamp/motion_planner/holding_prm/`. What is reused from the rest of the code base: `ssik_inprocess.solve` and `ssik_inprocess.fk` (the generated per-arm solvers and their forward kinematics), `ssik_ik.rebranch_toward_seed`, `ARM_SLICE` and `ARM_BASE_LINK`, and from the benchmark `load_trimmed_problem`, `open_scene`, `state_with_conf12` and `provenance`.

Kept apart from the BiRRT work on purpose. Other sessions were editing the planner core, the benchmark runner and the benchmark dashboard while this was written. The pilot therefore lives in new files only, writes only under `<root>/prm/` (never `runs/` or the batch catalogue `runs.json`), has its own command line, and never imports `dual_arm_task_space_rrt/core.py`: that module opens its log file in `"w"` mode when imported, which empties a log a running batch may be writing. Three places where the benchmark reaches the core indirectly are avoided by writing short local versions: the collision check (`SceneCtx.collision_fn` imports the core for its explain helper), the library loader (`benchmark.library` imports `benchmark.sampling`, which imports the core) and the static-site copy. A test runs the import in a fresh process and asserts no core module is loaded; the log file's size and time stamp were unchanged across every pilot run.

3 Building blocks

3.1 The holding violation

Algorithm 1: HoldingViolation — `holding_violation` / `violation_from_tools`

function HoldingViolation(q)
  $F_L \leftarrow W_L\,\mathrm{fk}_L(q_L)$; $F_R \leftarrow W_R\,\mathrm{fk}_R(q_R)$ ▷ W: arm base in the world, read once per scene
  $F_R^\star \leftarrow F_L\,G_L^{-1}G_R$ ▷ where the bar placed by the left flange wants the right one
  $F_L^\star \leftarrow F_R\,G_R^{-1}G_L$ ▷ and the other way round
  $e_p \leftarrow \max(\|p(F_R^\star) - p(F_R)\|\,,\ \|p(F_L^\star) - p(F_L)\|)$
  $e_r \leftarrow \max(\angle(F_R^\star, F_R),\ \angle(F_L^\star, F_L))$ ▷ angle by atan2(sin, cos)
  return (e_p, e_r)
end function

Three details. (i) fk is the ssik artifact's own forward kinematics, generated from the calibrated URDF like the solver; it takes about 30 μs and agrees with PyBullet to 3.4×10^{-7} on every transform entry (measured on 45 library configurations of B3). So the measure needs no simulator, and the parent process can score whole paths. (ii) Taking the larger of the two directions makes the measure independent of which arm is the reference: the two angles are equal, the two distances differ by the lever arm between the grasps. (iii) The angle of a nearly identity rotation is taken from both its sine and cosine; an arc-cosine alone loses about half the digits, which on the dashboard showed as a spurious 0.1° error at nodes that hold the bar exactly. On every library configuration tested (primary goals, goal branch pairs, starts) the measure returns 10^{-7} m and 2×10^{-7} rad, four hundred times below the production FK gate.

One number for a tolerance. A tolerance is one number t in millimetres. A configuration passes it when $e_p \le t\,\mathrm{mm}$ *and* $e_r \le \rho\,t$ degrees, with $\rho = 0.4°/\mathrm{mm}$ by default (the "5 mm and 2 degrees" pair of the pilot plan). Equivalently its *score*

$$s(q) = \max(1000\,e_p,\ e_r^\circ/\rho) \quad [\mathrm{mm}] \tag{2}$$

is at most t. Edges store e_p and e_r separately, so ρ and t can be changed without re-measuring anything.

3.2 Straight lines in joint space, and why continuity is free

Interp($q_a, q_b; \delta$) (`interpolate_confs`) returns the points of the straight joint-space line cut into $\lceil \|q_b - q_a\|_\infty / \delta \rceil$ equal pieces; the difference is taken on raw joint values, the same rule as the BiRRT's gate (`core.joint_step_deltas`). With $\delta \le \tau$, every piece of every edge would pass the production continuity gate, so no path the roadmap returns can contain a branch flip. The tests check the bound; every path report re-measures the largest step between its dense points.

3.3 Turns

Four of the six UR joint ranges are wider than one turn (shoulder pan $\pm2\pi$, shoulder lift ±4.71, wrists ±4.19), so one physical posture can be written as two or three joint vectors 2π apart. A roadmap that measures raw joint differences sees these "twins" a full turn apart, and an edge between them would be a full-turn swing. Rebranch(q, r) (`rebranch12`, which applies `ssik_ik.rebranch_toward_seed` per arm) moves every joint onto the turn nearest a reference r when that stays in limits. The pilot uses two references: the configuration *as given* (the library's own turn) and the *home* configuration. Anchors and endpoints are inserted on both turns when

they differ; the query reports which turn a path used. This matters in practice: for B18 goal 2, start 0's right shoulder pan sits at $-319°$ in the library, while the goal's branch pairs sit at $+50°$ or $+204°$; its home-turn twin is $+41°$, nine degrees from three of them.

3.4 Collision

The check is the production one: copy the template cell state, write the twelve joints, and let compas_fab test self-collision, the tools, the attached bar and the environment with the scene's allowed-collision lists (`make_collision_fn`; the same call as `api._build_cfab_collision_fn`). It costs about 15 ms and dominates everything else the pilot does: one ssik solve is 1.8 ms, one forward-kinematics pair 60 μs. A 2π twin is the same physical pose, so it is never checked twice.

4 Building the roadmap

4.1 Anchors

Draws are centred on *anchors* (`make_anchors`): the home configuration, every collision-free branch pair recorded for the chosen goals (`free_branch_pairs`), and a spread of library starts per goal (40 by default, picked across the three difficulty classes and the three carry anchors as the benchmark's `select_starts` does), each on its given and its home turn. Anchors are also inserted as nodes. A global box spanned by all anchors is available but off: the 81 anchors of B18 goal 2 span $214°$ to $580°$ per joint (the right shoulder pan the most), i.e. the full range, and draws from that box are the wound-up arms the pilot plan warned about.

4.2 Sampling: forward kinematics for one arm, IK for the other

Algorithm 2: DrawDriver — `draw_driver`

function DrawDriver(mode, driver)
  $a \leftarrow$ uniform anchor index; $c \leftarrow$ anchor$[a]$ ▷ c: the 12-joint centre of the draw
  if mode = `anchor_box` **then** $q_d \sim \mathcal{U}(c_d \pm m)$ ▷ $m = 90°$ per joint
  else if mode = `anchor_gauss` **then** $q_d \sim \mathcal{N}(c_d, \sigma^2)$ ▷ $\sigma = 30°$
  else if mode = `anchor_line` **then**
    $b \leftarrow$ an anchor of a different role (start $\leftrightarrow$ goal) ▷ a start–goal line when possible
    $c \leftarrow c + u\,(\text{anchor}[b] - c)$, $u \sim \mathcal{U}(0,1)$; $q_d \sim \mathcal{N}(c_d, \sigma_\ell^2)$ ▷ $\sigma_\ell = 10°$
  end if
  return (clip(q_d, limits), c, a)
end function

Algorithm 3: SampleChunk — `sample_chunk` / `lift_draw`

function SampleChunk(seed, k, n)
  rng $\leftarrow$ `SeedSequence`(seed, k) ▷ chunk k gives the same nodes on any worker
  for $i = 1 \ldots n$ **do**
    driver $\leftarrow$ L if $(kn + i)$ even else R; other $\leftarrow$ the other arm ▷ both arms drive equally often
    mode $\sim$ mix; $(q_d, c, a) \leftarrow$ DrawDriver(mode, driver) ▷ default mix 0.2 / 0.3 / 0.5
    $T \leftarrow \mathrm{FK}_{\text{driver}}(q_d)\, G_{\text{driver}}^{-1}$ ▷ the bar pose this arm implies [P-1]
    $B \leftarrow \mathrm{IK}_{\text{other}}(T\, G_{\text{other}};\ \text{seed} = c_{\text{other}},\ \text{rescue off})$ ▷ re-branched to c, nearest first [P-8]
    $B \leftarrow B[1{:}\beta]$ ▷ $\beta = 1$: the branch nearest the centre
    for $q_o \in B$ **do**

10: $q \leftarrow (q_d, q_o)$; **assert** HoldingViolation$(q) \le 10^{-4}$ ▷ a failure stops the build
11: **if not** Collides(q) **then** keep q with (mode, driver, a, geometry)
12: **end if**
13: **end for**
14: **end for**
15: **end function**

Keeping only the branch nearest the draw centre ($\beta = 1$) was a measured choice. With every branch kept, a 300-node smoke roadmap (bar B3) split into 103 components: the other arm's median distance to the anchor was 182°, i.e. most kept nodes were flipped elbows or wrists that sit far from every other node and join nothing. With $\beta = 1$ the same budget gave 44 components. The B18 comparison at 2000 nodes is in Table 6. Other branch sheets are still represented, because every branch pair of the goal is an anchor and draws around an anchor are seeded from that anchor.

4.3 Edges

Algorithm 4: EvaluateEdge — `evaluate_edge`

---

function EvaluateEdge(q_a, q_b)
  $(e_p^{\max}, e_r^{\max}, t^\star) \leftarrow$ max of HoldingViolation over Interp$(q_a, q_b; \delta_v)$ ▷ $\delta_v = 2.5°$, nearly free [P-3]
  $P \leftarrow$ interior points of Interp$(q_a, q_b; \delta_c)$ ▷ $\delta_c = 10° = \tau$ [P-2]
  for j **in** bisection order of P **do** ▷ middle first, then quarters, …
    if Collides(P_j) **then return** (free = false, $t_{\text{collide}} = j/|P|$, $e_p^{\max}$, $e_r^{\max}$)
    end if
  end for
  return (free = true, $e_p^{\max}$, $e_r^{\max}$, $t^\star$, $\|q_b - q_a\|_2$, $\|q_b - q_a\|_\infty$)
end function

Candidate pairs (`candidate_pairs`) are each node's $k = 10$ nearest neighbours in the largest-joint-move metric (a `cKDTree` with $p = \infty$) no further than 90°, symmetrised and sorted so the work splits into the same chunks every time. The violation is measured four times more finely than collision because it costs two forward-kinematics calls per point; collision is checked at the continuity step, which is also what the production gate would look at. The end nodes are already known to be free.

Algorithm 5: BuildRoadmap — `build_roadmap`

---

function BuildRoadmap(bar, goals, N)
  start the worker pool (one scene per process) ▷ Section 6
  anchors ← `make_anchors`(goals); insert the collision-free ones as nodes
  while fewer than N sampled nodes kept **do**
    issue chunks $k, k{+}1, \ldots$ of SampleChunk in parallel ▷ round size from the measured yield
  end while
  keep the first N sampled nodes in chunk order ▷ same nodes for any worker count
  pairs ← `candidate_pairs`(nodes, k, radius)
  edges ← EvaluateEdge on every pair, in parallel (collision skipped with `-lazy`)
  save `roadmap.json.gz` and `config.json`; add the run to `prm/index.json`
end function

---

5 Querying the roadmap

5.1 Endpoints

Algorithm 6: ExpandBarPose — `expand_bar_pose`

function ExpandBarPose(T, references)
  $B_L \leftarrow \mathrm{IK}_L(TG_L)$; $B_R \leftarrow \mathrm{IK}_R(TG_R)$ ▷ all branches, rescue on (one-off solve)
  for $(q_L, q_R) \in B_L \times B_R$ **do** ▷ up to 64 pairs [P-5]
    if Collides(q_L, q_R) **then**
      continue
    end if
    for (name, r) **in** references **do** ▷ given: the library configuration; home
      $q' \leftarrow$ Rebranch($(q_L, q_R), r$); merge with an equal q' already listed, adding the name
    end for
  end for
  return every distinct q' with its pair index and turn names
end function

The goal pose and every chosen start pose are expanded in parallel, one pose per task. Each start node carries its start index, carry anchor, library difficulty, branch-pair index, turn, and whether it is the library's own configuration. The expansion must find the library's `free_branch_pairs` again (they are the same pairs, solved by the same solver); a test checks this on the smoke goal, every query records the count, and every B18 query found all 7 of goal 2's pairs.

5.2 The gap of every node

Algorithm 7: MinimaxToGoals — `minimax_to_goals`

function MinimaxToGoals($\mathcal{G}$, goal set)
  weight every usable edge by its score s (Eq. 2); join a sink to every goal node at 0
  $M \leftarrow$ minimum spanning tree of $\mathcal{G}$ (Kruskal)
  gap(sink) $\leftarrow 0$
  for each tree edge (u, v) in breadth-first order from the sink **do**
    $\mathrm{gap}(v) \leftarrow \max(\mathrm{gap}(u), s_{uv})$; $\mathrm{parent}(v) \leftarrow u$
  end for
  return gap, parent ▷ gap $= \infty$ for nodes not joined to any goal
end function

In a minimum spanning tree, the tree path between two nodes has the smallest possible largest edge of any path between them. With the sink joined to the goal set, one tree gives *every* node its *gap*: the smallest tolerance at which a chain of free edges joins it to some goal node, and the next node on that chain. The best start is the start node with the smallest gap; a start index's gap is the smallest gap over its branch pairs and turns. On the dashboard any clicked node's best route to the goals is simply followed through the parent pointers.

5.3 Lazy collision checking

Expanding every start of a goal gives thousands of endpoint nodes and tens of thousands of candidate endpoint edges; checking all of them for collision would cost more than the roadmap. The query

measures the holding error of every endpoint edge (cheap) but checks collision only where it decides an answer.

Algorithm 8: VerifyGaps and LazySearch — `query_roadmap` / `lazy_search`

function VerifyGaps
  repeat
    (gap, parent) ← MinimaxToGoals with unchecked edges assumed free [P-7]
    U ← unchecked edges on the route (via parent) of each start index's best node
    collision-check U in parallel; mark the colliding ones blocked
  until $U = \varnothing$
end function

function LazySearch(find_path)
  repeat
    P ← find_path(edges free or unchecked) ▷ filtered or penalised Dijkstra, source → sink
    U ← unchecked edges of P; check them in parallel; mark the colliding ones
  until P = None **or** $U = \varnothing$
  return P
end function

Both loops are exact, not approximations. Assuming unchecked edges free can only lower a gap or shorten a path; when the deciding route comes back with every edge checked and free, nothing cheaper exists in the true graph, since the true graph is a subset of the optimistic one. For the best node of each start index this makes the gap exact; the gaps of other nodes stay optimistic lower bounds and are only used for colouring.

5.4 The questions asked

Algorithm 9: QueryRoadmap — `query_roadmap`

function QueryRoadmap(roadmap, goal, starts, tolerances)
  goal nodes ← ExpandBarPose(T_g, {given: primary, home}) [P-5]
  start nodes ← $\bigcup_s$ ExpandBarPose(T_s, {given: q_s, home}); reuse roadmap nodes at the same q
  endpoint edges ← EvaluateEdge without collision on each new node's k nearest (radius 90°)
  VerifyGaps() ▷ per-start gap, exact
  best ← start node of smallest gap; report its route (the minimax route)
  report the shortest path within the best gap (LazySearch, filtered at gap)
  for t **in** tolerances $(2, 5, 10, 30\,\text{mm})$ **do** [P-6]
    filtered: LazySearch over edges with $s \le t$, weight $= \|\Delta q\|_2$
    penalised: LazySearch over all free edges, weight $= \|\Delta q\|_2 + w \max(0, s/t - 1)$
    starts connected ← #{start indices with gap $\le t$}
  end for
  save the query: endpoint tables, final edge verdicts, every node's gap and parent, the paths with their dense profiles
end function

Each path report stores the node list, the start and goal it used (start index, carry anchor, branch pair, turn; goal pair, turn), the dense joint path at δ_v, the holding error at every dense point, where

the worst point sits, and the largest joint step between dense points (at most δ_v by construction). The penalised weight uses $w = 10$: going one tolerance over costs as much as ten radians of joint travel.

6 Running it in parallel

Sampling, edge checking and endpoint expansion are the same small computation thousands of times over, each independent. `WorkerPool` is a `ProcessPoolExecutor` with the `spawn` start method (the pattern of the benchmark's parallel library build); each worker opens its own headless PyBullet scene once, in its initializer, and keeps it. Processes, not threads: `pybullet_planning` keeps one global simulation and the collision check writes joints into it. Tasks and results are plain lists, so nothing that cannot cross a process boundary on Windows ever does; each edge task carries the configurations it needs (64 edges, about 12 kB). Results are re-sorted by chunk index, so the roadmap is the same for any worker count, and the parent never opens a scene: it does the nearest-neighbour search, the graph searches and the file writing. A worker that dies surfaces as a broken pool at once. Each worker sets `OMP_NUM_THREADS=1` so six or eight of them do not fight a running benchmark batch for cores; the default is six workers, eight were used for the runs below while no batch was running.

7 Storage and the dashboard

file	contents
`prm/index.json`	every roadmap run (only this tool writes it)
`prm/<run>/config.json`	settings, anchors, tallies per phase, timings, provenance (git state of every repository, versions, ssik artifacts)
`prm/<run>/roadmap.json.gz`	holding model; nodes as columns (configuration, both flange positions, their midpoint, bar pose, violation, kind, sampling mode, driver, anchor); edges as columns (ends, lengths, checked, free, where it collides, worst position and rotation error, where)
`prm/<run>/queries/<tag>.json.gz`	endpoint nodes and edges, the final verdict of every edge, every node's gap and parent, the paths with dense profiles, the per-start table
`prm/<run>/dashboard/`	the page (`prm.html`, also as `index.html`) with the shared scripts

The page reuses the benchmark dashboard's robot viewer, loaders and charts without changing them. Each node is a ball at the midpoint of the two flanges (grey sampled, purple anchors, green starts, blue goals; or coloured by gap). Balls are instanced meshes registered in the viewer's list of pickable clouds, so the existing click handler reports them. Clicking a ball poses the robot, lists the twelve joints, the node's role (anchor, which start or goal pair, which turn), its gap, and its edges sorted by holding error, each with a "walk" button. Edges are drawn green within the tolerance slider's value, amber up to three times it, red beyond. The best route of the clicked node is drawn and can be stepped through; at every step the page recomputes the holding error from the posed flanges, and a second, orange bar placed by the right flange shows the drift. Serve the data root and open `http://localhost:8000/prm/<run>/dashboard/`.

How to run it. From the repository root, with `HUSKY_BENCH_ROOT` set to the benchmark data root (PowerShell shown):

```
.venv\Scripts\python.exe scripts\prm_pilot.py build --name B18_g2_n4000 --library cdfm_v1 `
    --bar B18 --goals 2 --anchor-starts 40 --nodes 4000 --workers 8
.venv\Scripts\python.exe scripts\prm_pilot.py query --name B18_g2_n4000 --goal 2 `
    --starts all --tag all
.venv\Scripts\python.exe scripts\prm_pilot.py export --name B18_g2_n4000
.venv\Scripts\python.exe scripts\prm_pilot.py info --name B18_g2_n4000
python -m http.server --directory $env:HUSKY_BENCH_ROOT 8000
```

`-starts` also takes a class list (`free,blocked`) or start indices (`0,12`). Long runs should be started detached (e.g. `Start-Process` with the output sent to a log), not from a Claude Code background shell, which stops after ten minutes.

8 Core differences from the task-space BiRRT

P-1 Nodes are joint configurations that hold the bar.
The BiRRT's nodes are bar poses with an IK side cache (its D-1). Here a node *is* twelve joint values; the bar pose is derived from it. It is made by driving one arm and solving the other, so it holds the bar exactly.

P-2 No continuity gate.
Edges are straight joint-space lines cut at τ, so continuity holds by construction. The gate that stops 75 % of the BiRRT's extensions (D-6b) does not exist.

P-3 The holding constraint is soft along edges.
It is measured at every 2.5° and becomes the edge's score; the search either filters by a tolerance or pays a penalty above it.

P-4 Distance is the largest joint move.
Nearest neighbours use the raw joint ∞-norm, not the bar-corner feature metric (D-2): what matters for an edge is how far the joints move.

P-5 Sets, not points.
The start and goal are sets of every collision-free branch pair on two turns. The goal/home branch pairing heuristic and the requirement that both endpoints be fixed in advance (D-9) disappear; the search chooses.

P-6 One graph, many questions.
The graph keeps every alternative and is searched again under any tolerance or weight; there are no restarts (D-8) and no trees thrown away.

P-7 Collision checked where it decides the answer.
Endpoint edges are checked lazily, and the loops that do so are exact.

P-8 IK only at nodes.
One analytical solve per draw and 64 pairs per endpoint pose; none along any edge. The BiRRT solves both arms at every waypoint of every extension.

P-9 Parallel by design.
Sampling, edges and endpoints are split across worker processes, each with its own scene.

P-10 The answer is a number.
Every start gets a gap (the smallest tolerance at which it connects), so "near-feasible" and "disconnected" can be told apart.

9 Parameters and their defaults

parameter (flag)	default	meaning
`-nodes`	2000	sampled nodes kept (anchors come on top)
`-goals`, `-anchor-starts`	all, 40	goals whose pairs are anchors; library starts per goal used as anchors
`-anchor-turns`	both	anchors as given and on the home turn
`-mode-mix`	0.2/0.3/0.5	`anchor_box` / `anchor_gauss` / `anchor_line`
`-box-margin-deg`, `-gauss-sigma-deg`, `-line-sigma-deg`	90, 30, 10	spreads of the three draws
`-max-branches`	1	other-arm branches kept per draw, nearest the centre
`-k`, `-radius-deg`	10, 90	neighbours per node; longest edge (largest joint move)
`-collision-step-deg`	10	collision checks along an edge ($= \tau$)
`-viol-step-deg`	2.5	holding-error points along an edge
`-rot-deg-per-mm`	0.4	ρ in Eq. 2
`-k-endpoint`, `-radius-endpoint-deg`	10, 90	endpoint edges
`-tolerances-mm`, `-w-hold`	2,5,10,30; 10	query tolerances; penalty weight
`-endpoint-check`	lazy	collision of endpoint edges
`-workers`	6	worker processes
`-seed`	0	random seed

10 What the pilot measured

10.1 What was run

All runs used the benchmark's library `cdfm_v1` and its trimmed scenes (the scene key and git state are in each run's provenance record), the calibrated ssik solvers, eight worker processes on the 16-thread workstation (no benchmark batch was running), and the defaults of Section 9 unless stated.

- **B18, goal 2** — the goal whose 807 surveyed starts the BiRRT was run on in full (experiment 2, batch `20260929_1756`: two seeds per start, 21 of 807 starts solved at least once). Roadmaps with 500, 1000, 2000 and 4000 sampled nodes, all anchored on goal 2 with 40 anchor starts (81 anchors in all). Each was queried twice: starts 0 and 12 only (night 1: start 0 solved by 0 of 10 seeds, start 12 by 6 of 10), and all 807 starts.
- **B18, 2000 nodes, variations** — seeds 1 and 2, every IK branch kept per draw, and no start–goal line draws.
- **B3, goals 4, 11 and 16** — one 2000-node roadmap anchored on all three goals, queried per goal with the three night-1 starts.

Run folders are `<root>/prm/B18_g2_n<budget>/`, `B18_g2_n2000_<variation>/` and `B3_g4-11-16_n2000/`; the logs are in `<root>/prm/logs/`.

10.2 How fast a joint-space line leaves the holding surface

Figure 1 is the single most important measurement of the pilot, because it fixes how dense a roadmap has to be. Both ends of every edge hold the bar exactly, yet the straight line between them does not: the median worst error is 7.3 mm for edges shorter than 20° and 125.2 mm for edges of 60° and more. Finer bins give the numbers that matter for a tolerance: edges of 2.5–7.5° have a median worst error of 1.0 mm (51 edges), edges of 7.5–12.5° (the continuity step τ) 3.6 mm (642 edges), and edges of 12.5–17.5° 6.9 mm (2683 edges). Over this range the error grows roughly in proportion to the length rather than with its square, so the roadmap's edges are far from the regime where a straight chord hugs the curved holding surface. Two consequences follow. First, a sub-2 mm path needs a chain of edges of about 5° each; a roadmap sampled freely in twelve dimensions does not supply that density, so the gaps below are best read as *upper bounds* on how close a start is to the goal set. Second, the holding error, not collision, is what limits a joint-space roadmap: 93 % of

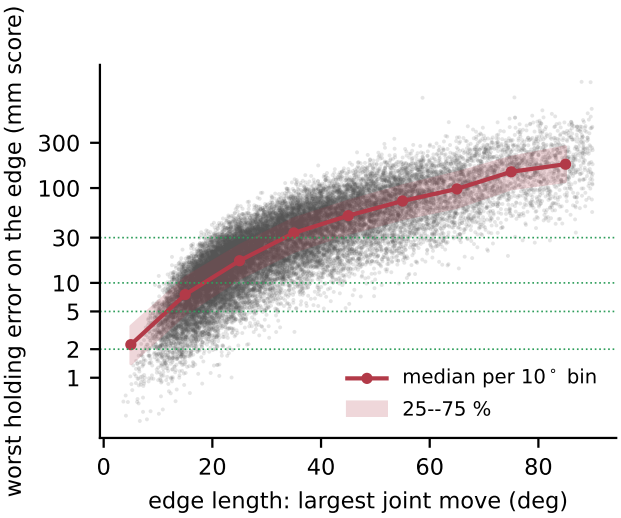


Figure 1: Worst holding error (score, Eq. 2) of every free edge of the 4000-node B18 roadmap against the edge's largest joint move. Dotted lines: the four query tolerances.

candidate edges are collision-free, but only 1.1 % of those free edges stay within 2 mm, 7.5 % within 5 mm, 21 % within 10 mm and 57 % within 30 mm.

10.3 The roadmaps and what they cost

Table 1: The B18 roadmaps (goal-2 anchors, seed 0, 8 workers). Draws are driver-arm draws, one IK call each; collision checks count sampling and edges together. The anchor columns give how many of the goal and start anchors sit in the largest connected component of the free graph. Times are wall-clock seconds; the last column is worker-seconds over wall-seconds for the edge phase.

budget	nodes	draws	IK	coll.	edges	free	comp.	largest	goal	start	sample	edges	total	speed-up
500	581	2656	2656	11818	2922	2657	71	262	3/7	24/73	12	28	51	7.4
1000	1081	5344	5344	23969	6094	5586	90	525	3/7	25/73	16	74	100	7.8
2000	2081	10688	10688	47606	13028	12000	125	1285	7/7	27/73	24	103	138	7.9
4000	4081	21408	21408	89047	26908	25056	173	2557	7/7	27/73	40	181	232	8.0

Counting queries, as the meeting asked. Every build cost one analytical IK call per draw, and the sampler kept about one node in four and a half draws: of the 21 408 draws of the 4000-node build, 56 % were reachable by the other arm, and 39 % of those were collision-free. The line mode was by far the most productive (31 % of its draws kept, against 17 % for the Gaussian and 5 % for the box), which is why it carries half the mix. The edge phase then dominates: 77 076 collision checks for 26 908 edges, of which 1852 (6.9 %) collide. In worker time, collision checking was most of the whole build (93 %: 1571 of 1680 worker-seconds at 4000 nodes, 15.7–17.9 ms per check) and IK 2.7 % (2.1 ms per call); the 382 241 holding-error points cost nothing measurable. This inverts the pilot plan's expectation that IK would be the expensive operation: here it is the full-cell compas_fab check, exactly as in the BiRRT's own profile. The GPU question is therefore a question about batched collision checking, not about IK.

Parallel speed-up. Edge checking ran at 7.4–8.0 worker-seconds per wall-second on eight workers (Table 1), i.e. essentially linearly; sampling reached 6.0 at 4000 nodes and less on small budgets, where a round is only a few chunks. The query's endpoint expansion (38 384 branch pairs collision-checked for the 807 starts) ran at 7.2 and the endpoint edges at 7.7. The whole 4000-node build

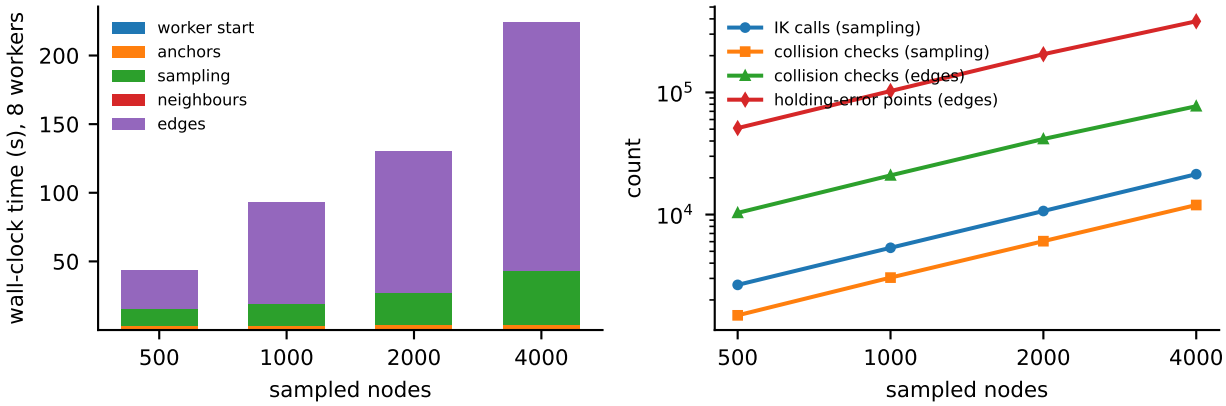


Figure 2: Left: where the build time goes. Right: the work counted, against the node budget.

took 232 s of wall time; one all-starts query took 208–229 s whatever the roadmap size, dominated by the endpoint expansion (73 s) and the lazy gap verification (104 s, 80 rounds, 5461 of the 99161 endpoint edges checked, 1231 of them blocked).

Lazy checking is exact in practice too. On the smoke roadmap (bar B3, 333 nodes, 380 starts) the same query was run with endpoint edges checked lazily and eagerly: the per-start gaps agree for all 380 starts, and the lazy query took 151 s against 593 s (two workers).

10.4 Starts 0 and 12, and all 807 starts, against the node budget

Table 2: What each B18 roadmap answers (gaps in mm score). "lib." is the gap of the start's own library configuration alone; "best" is the best over all of the start's branch pairs and both turns. ∞: no route of free edges to any goal node. The right half is the query over all 807 starts: the best gap, how many starts reach the goal set at all, how many do so within each tolerance, and the query's wall time.

	start 0		start 12		all 807 starts						
budget	best	lib.	best	lib.	best gap	reach	≤2	≤5	≤10	≤30	time (s)
500	82.6	∞	7.9	22.1	7.9	650/807	0	0	13	114	208
1000	41.8	∞	5.7	29.2	5.7	657/807	0	0	17	115	208
2000	41.4	∞	5.1	29.2	4.2	657/807	0	1	24	171	229
4000	21.3	∞	4.5	24.2	2.9	657/807	0	11	36	462	212

Three things stand out in Table 2 and Fig. 3.

Start 12 is near-feasible, start 0 is not. Start 12, which the BiRRT solved with 6 of 10 seeds in night 1, reaches the goal set within 7.9 mm on the smallest roadmap and within 4.5 mm on the largest. Start 0, solved by none of 10 seeds, sits at 82.6 mm on 500 nodes and still 21.3 mm on 4000 nodes. The reason is local: on the 2000-node roadmap, the 11–20 sampled nodes drawn around start 0's two anchors ended up a median 96–121° away from it (their other arm lands on a distant IK branch, or they collide), and only two nodes lie within 30° of it; around start 12 the same numbers are 71–73° and 13–28 nodes. Start 0's gap on that roadmap is set by three long edges within the first five steps out of it (45–52°, 31–41 mm), not by anything near the goal.

The library's own start configuration is often the wrong one. For both starts the gap of the library configuration alone (column "lib.") is much larger than the best over all branch pairs and turns; for start 0 it is unreachable at every budget. Over all starts, a different branch pair or turn beat the library configuration for 610 of the 657 reachable starts, and the best node sat on the home turn for most of them (Table 3). This is the quantitative form of the meeting's worry about

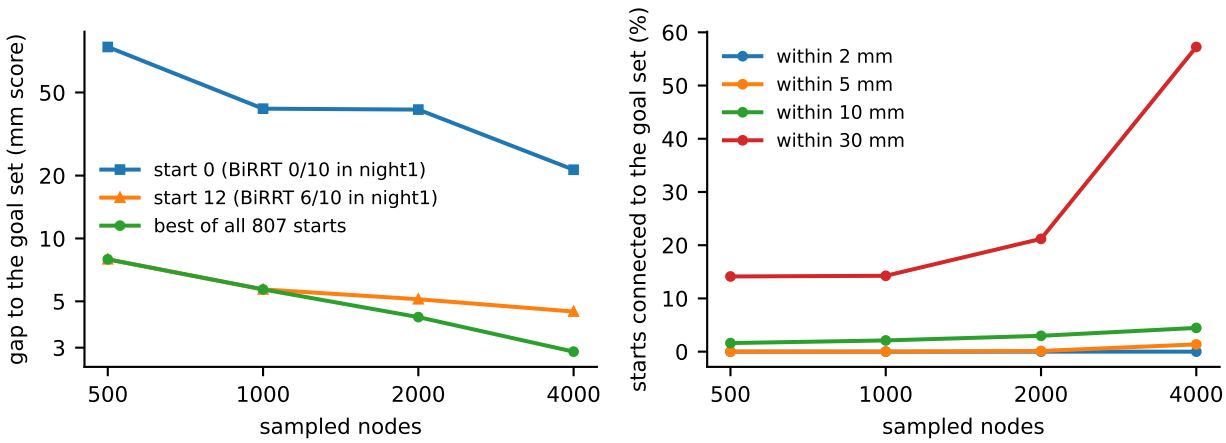


Figure 3: Left: gap to the goal set against the node budget for start 0, start 12 and the best of all starts. Right: share of the 807 starts that reach the goal set within each tolerance.

start/goal branch pairing: fixing the start configuration before searching throws away most of the useful connections.

More nodes help, slowly. The best gap over all starts falls from 7.9 to 2.9 mm from 500 to 4000 nodes, the median over starts from 49 to 24 mm, and the number of starts within 30 mm from 114 to 462 (the least stable of these numbers: Table 6 shows it ranging from 171 to 405 across seeds at 2000 nodes). No start reaches the goal set within 2 mm at any budget. Exactly the same 150 starts are unreachable at 1000, 2000 and 4000 nodes: their start nodes do have edges, but into clusters of the roadmap that no edge of at most 90° joins to the goal's component (the 4000-node roadmap has 173 components); 136 of them are "broken" starts.

Table 3: The turn of each reachable start's best node (4000-node roadmap).

turn of the best node	starts
given	76
home	496
given and home	85

10.5 Which starts are near the goal: classes, carry anchors, and the BiRRT

Table 4: All 807 starts of B18 goal 2 on the 4000-node roadmap, by the library's straight-line class and by the BiRRT's outcome in experiment 2. "reach": starts with any route to the goal set; the last four columns count starts within each tolerance (mm).

group	starts	reach	median gap	≤2	≤5	≤10	≤30
free	28	28	6.8	0	7	19	28
blocked	43	29	11.5	0	4	11	26
broken	736	600	24.4	0	0	6	408
all	807	657	24.3	0	11	36	462
BiRRT solved (exp. 2)	21	21	6.8	0	8	15	21
BiRRT never solved	786	636	24.3	0	3	21	441

The roadmap ranks the starts the way the BiRRT did. The roadmap never ran the BiRRT and knows nothing about its results, yet the starts the BiRRT solved at least once in experiment 2 are the ones the roadmap puts closest to the goal set: all 21 of them are reachable, their median

Table 5: The same starts by carry anchor. The last column counts the starts of each anchor that the BiRRT solved at least once in experiment 2.

carry anchor	starts	reach	median gap	≤10	≤30	BiRRT solved
back	306	195	23.4	36	171	21
horizontal	458	419	24.3	0	291	0
vertical	43	43	57.0	0	0	0

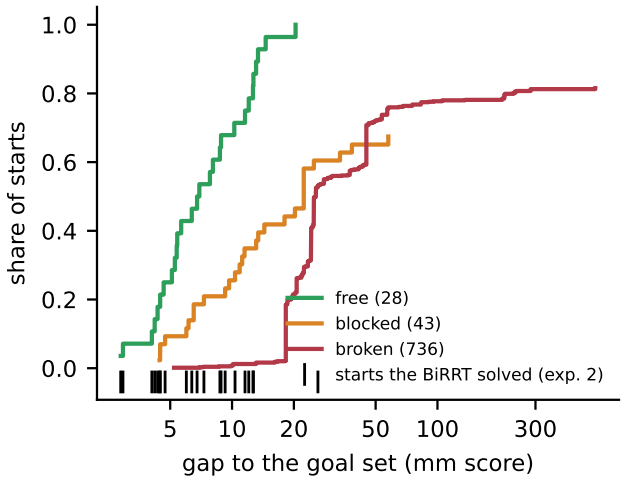


Figure 4: Share of starts whose gap is below a value, per library class (starts with no route are the missing share at the right). Ticks under the axis: the gaps of the starts the BiRRT solved in experiment 2.

gap is 6.8 mm against 24.3 mm for the starts it never solved, 15 of them are within 10 mm and all 21 within 30 mm (Table 4). The five starts with the smallest gaps were all solved by the BiRRT, and seven of the best ten. The library's straight-line classes are ordered the same way (medians 6.8, 11.5 and 24.4 mm for free, blocked and broken). The ranking is not perfect in the other direction: 21 of the 36 starts within 10 mm were never solved in experiment 2, but that batch ran only two seeds per start, and one of those 21 is start 12, which night 1 solved with 6 of 10 seeds.

The carry anchor matters most. Table 5 is the clearest single result. Every start the BiRRT solved is a "back" carry pose, and so is every start within 10 mm of the goal set; no horizontal or vertical start comes within 10 mm, and the vertical ones have a median gap of 57 mm. The benchmark's night-1 analysis found that production's start derivation always picks the horizontal anchor and succeeds 2.7 % of the time from it. For this goal the roadmap says why: from a horizontal carry pose no route of short joint-space steps reaches any of the goal's branch pairs.

10.6 The best path

The best route of the 4000-node roadmap starts at start 117 (a back, free-class start the BiRRT solved), on branch pair 38 and the home turn — not its library configuration — and ends at the goal's branch pair 7 (library numbering). It has 14 nodes and travels 4.57 rad in joint space, with no joint moving more than 2.47° between the dense points of the profile, so it contains no branch flip by construction. Its worst holding error is 2.87 mm. Figure 5 shows what that number is made of: the error rises from zero at each node to a peak in the middle of each edge and falls back to zero at the next node, and the peaks are all of similar height. Only two of the thirteen edges peak above 2 mm (at 2.2 and 2.9 mm); the others peak between 0.8 and 1.9 mm. There is no plateau and no single dominant spike: this is a resolution limit of the roadmap, not a sign that the start and the goal sit on disconnected sheets. Replacing those two edges by shorter ones — locally resampling

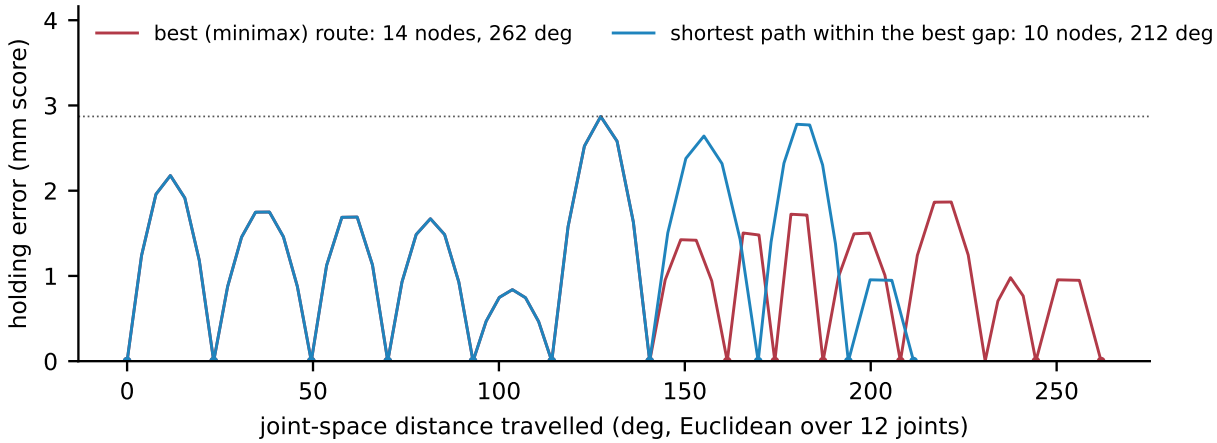


Figure 5: Holding error along the best route of the 4000-node roadmap (start 117) and along the shortest path within the same gap. Dots: roadmap nodes, where the error is zero. Dotted line: the gap.

around them, as the plan's refinement step proposes — is all that stands between this route and a sub-2 mm path. The filtered searches tell the same story from the other side: within 5 mm eleven starts connect, the shortest such path (7 nodes, from start 312, another BiRRT-solved start) has a worst error of 4.6 mm, and within 10 and 30 mm the shortest paths have 5–6 nodes.

10.7 Seeds and two sampler choices

Table 6: The 2000-node B18 roadmap under three seeds and two sampler changes. Gaps in mm; "reach", "≤10" and "≤30" count the 807 starts; build time in s.

roadmap	draws	edges	comp.	largest	start 0	start 12	best	reach	≤10	≤30	median	build
seed 0 (default)	10688	13028	125	1285	41.4	5.1	4.2	657	24	171	30.5	138
seed 1	9280	12983	139	1044	33.0	3.9	3.8	651	25	400	24.3	114
seed 2	10304	12896	148	1034	23.5	5.0	5.0	658	27	405	24.5	133
every IK branch kept	2272	9500	296	755	73.2	5.3	5.2	658	16	112	44.0	110
no start–goal line draws	16768	11929	200	1460	38.2	13.7	13.7	650	0	406	25.5	132

Seeds. Across three seeds at 2000 nodes the best gap is 3.8–5.0 mm, start 12's gap 3.9–5.1 mm, and 24–27 starts are within 10 mm: these numbers are stable. Start 0's gap (23.5–41.4 mm) and the count within 30 mm (171, 400, 405) are not; seed 0 happens to be the unlucky one for both, so the 30 mm counts of Table 2 should be read with a spread of that size.

Keeping every IK branch makes each draw yield more nodes (only 2272 draws were needed for 2000 nodes, against about 10 000), but the nodes are scattered: the roadmap falls into 296 components, has 3500 fewer edges, and every quality number is worse (median gap 44 mm, 16 starts within 10 mm, 112 within 30 mm). This confirms on B18 what the smoke roadmap showed.

Without the start–goal line draws the roadmap still covers the space (406 starts within 30 mm, similar to the default seeds) but loses all precision: no start comes within 10 mm, and the best gap rises to 13.7 mm. The near-feasible routes of Fig. 5 are made almost entirely of line-draw nodes; they are what the roadmap's precision rests on. Line draws are also the most productive per draw (31 % kept, against 17 % for the Gaussian and 5 % for the box, at 4000 nodes), which is why the no-line roadmap needed 16 768 draws for the same 2000 nodes.

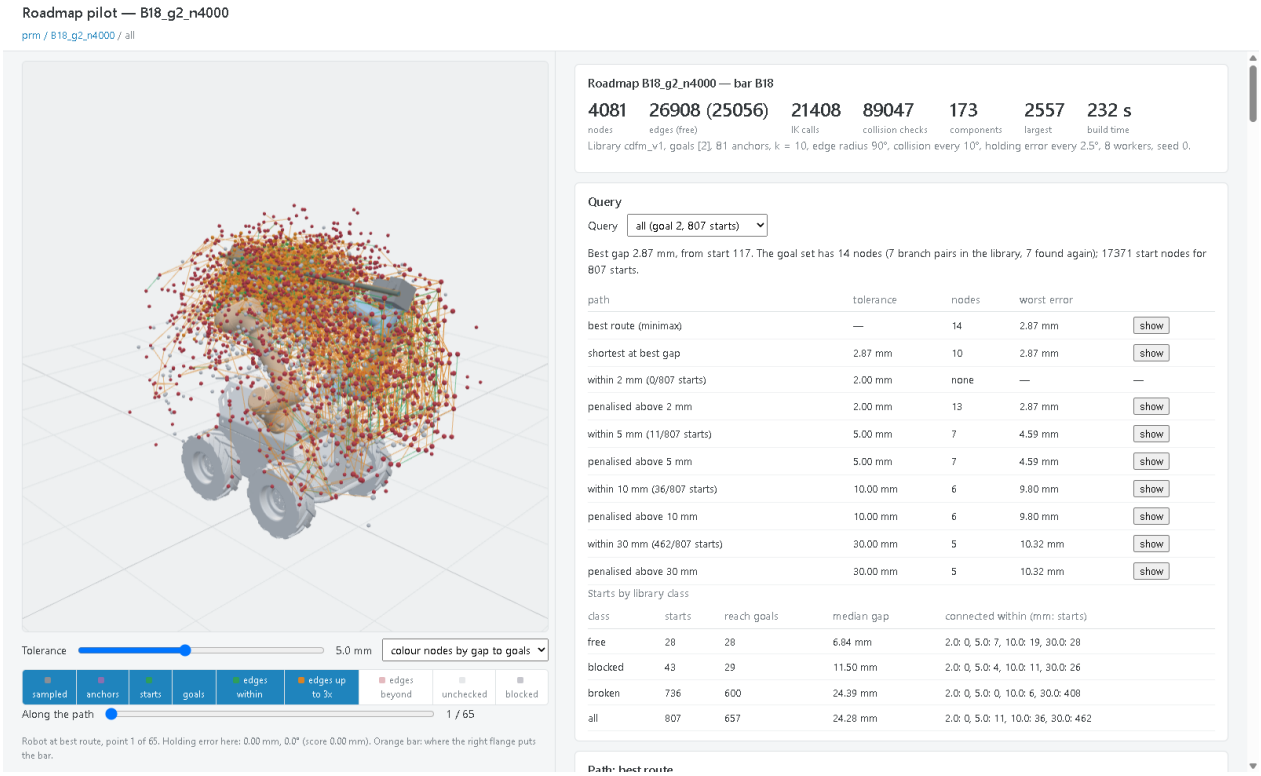


Figure 6: The roadmap page on the 4000-node B18 roadmap and the all-starts query: nodes at the midpoint of the two flanges, coloured by their gap to the goal set at the 5 mm slider setting, edges coloured by their worst holding error, the robot posed at the first point of the best route.

10.8 Bar B3

Table 7: Bar B3, one 2000-node roadmap anchored on goals 4, 11 and 16; the three night-1 starts of each goal. "night 1": seeds (of 10) the BiRRT solved.

goal	start	carry	class	night 1	gap	lib. gap	best pair
4	0	horizontal	broken	0/10	∞	∞	44
4	13	back	blocked	4/10	16.2	25.4	63
4	42	back	free	8/10	6.5	14.9	51
11	0	horizontal	blocked	0/10	16.7	16.7	44
11	13	back	free	2/10	7.6	35.6	46
11	221	horizontal	free	2/10	16.7	35.6	45
16	0	horizontal	broken	0/10	17.1	17.1	44
16	13	back	free	8/10	7.5	11.7	46
16	23	horizontal	free	3/10	17.1	17.1	47

On bar B3 the roadmap orders the three night-1 starts of each goal as the BiRRT did (Table 7). In goal 4, start 42 (8 of 10 seeds) has a 6.5 mm gap, start 13 (4 of 10) 16.2 mm, and start 0 (0 of 10) no route at all. In goal 16 the best-solved start (13, 8 of 10 seeds) has the smallest gap, 7.5 mm, against 17.1 mm for the other two. In goal 11, where starts 13 and 221 were each solved twice, start 13 has 7.6 mm and start 221 16.7 mm. Start 0 — production's horizontal pick in all three goals — has the largest gap or ties for it. Again the library configuration is rarely the best: start 13 of goal 11 reaches the goals within 7.6 mm on its best branch pair but only within 35.6 mm on its library configuration.

10.9 What this says, and what it does not

Observations.

(i) **The pilot's two questions have first answers.** (1) On B18 goal 2, a joint-continuous path exists whose holding error stays under 2.87 mm, from a start the BiRRT also solved, found with 4000 nodes: 23 024 IK calls (21 408 to build, 1616 to solve the 808 endpoint poses) and 141 501 collision checks (89 047 to build, 52 454 for the query over all 807 starts), in 232 s plus 212 s on eight workers. No path under 2 mm was found at any budget. (2) Where the roadmap fails, the failure is local and of known shape: the best route's error is a row of similar peaks at 1–3 mm, two edges over 2 mm; start 0's is a few long edges right next to the start.

(ii) **The gap is a useful difficulty measure without running the BiRRT.** It separates the starts the BiRRT solved from those it did not (median 6.8 against 24.3 mm), orders the library's classes, and on B3 orders the night-1 starts of three goals as the BiRRT did.

(iii) **The carry anchor, and the branch pair, are decided wrongly today.** For B18 goal 2 only back-carry starts come within 10 mm, while production picks the horizontal anchor; and for 610 of 657 reachable starts some other branch pair or turn beats the library's own start configuration. A roadmap query over a set of starts makes both choices by search.

(iv) **Global density is the wrong lever.** Doubling the node budget from 2000 to 4000 lowered the best gap from 4.2 to 2.9 mm and did not reach any start that was unreachable at 1000. The holding error of an edge is set by its length (Fig. 1), and random nodes do not shorten the edges of one particular route fast enough.

(v) **Collision checking is the cost.** 93 % of build worker time; IK is under 3 %. The work parallelises almost perfectly on the CPU (8.0x on 8 workers for edges). A GPU, if ever, would be for batched collision checks.

What the numbers do not say. A gap is an upper bound on the smallest holding error a path needs: a denser or better-placed roadmap can only lower it. "Unreachable" means unreachable in this roadmap within 90° edges, not infeasible. Each budget was built once (seed 0); at 2000 nodes a different seed moves the best gap by about 1 mm and the 30 mm count by more than a factor of two (Table 6). Collision along an edge is checked every 10° of joint motion, coarser than the production planner's 2 mm pose steps, so a roadmap path has to be re-solved exactly (`core.repair_joint_path`, step 7 of the plan) before it can be executed; the pilot does not do this yet. And all of this is one goal of one bar, plus three goals of B3.

Next steps suggested by the data.

1. **Refinement** (plan step 6): around the best route, sample densely with a small spread (a few degrees), or split its over-tolerance edges at their worst point and project the split onto the holding surface by solving the other arm, then search again. The best route here needs two edges fixed.
2. **A local sampling mode at the endpoints**: draws around an anchor currently land 70–120° away from it. A mode with a spread of a few degrees around starts and goal pairs would give start 0 the nodes it lacks.
3. **Bridge to production** (plan step 7): convert the best route to bar poses and re-solve it with `core.repair_joint_path` and the full collision check at production resolution.
4. **Try the back anchor in the BiRRT**: `start_home_anchor=back` for goals like this one is a one-flag experiment that the carry-anchor result predicts should help.

11 Reading guide

Line numbers are from the tree as written on 30 September 2026.

read	at	this report
`violation_from_tools`, `holding_violation`	`holding.py:295`, `:320`	Alg. 1
`pose_gap`	`holding.py:130`	the angle
`interpolate_confs`, `rebranch12`	`holding.py:387`, `:421`	continuity; turns
`make_collision_fn`	`sampling.py:68`	collision
`arm_branches`	`sampling.py:148`	IK on the cached base
`draw_driver`, `lift_draw`, `sample_chunk`	`sampling.py:278`, `:323`, `:362`	Algs. 2, 3
`expand_bar_pose`	`sampling.py:413`	Alg. 6
`candidate_pairs`, `evaluate_edge`, `edge_cost`	`roadmap.py:55`, `:163`, `:209`	Alg. 4
`minimax_to_goals`, `lazy_search`	`search.py:169`, `:222`	Algs. 7, 8
`WorkerPool.run`, `init_worker`	`parallel.py:229`, `:56`	Section 6
`make_anchors`, `build_roadmap`	`pilot.py:289`, `:409`	Alg. 5
`query_roadmap`	`pilot.py:617`	Alg. 9